Knowledge Graphs 20260424

KG Inferenz

- Inferenz in einem KG führt zu neuem Wissen in Form von neuen Edges im Graphen.
- In einem KG erfolgt die Inferenz durch Axiome und Regeln.
- Wir behandeln 3 Arten:

1) Klassen-Unterordnung (Class-Subsumption):
   → Wenn die Klasse A eine Unterklasse der Klasse B ist ($A \subseteq B$),
   dann is jede Instanz von A auch eine Instanz von B.

2) Eigenschaftskette (Property Chains):
   → Wenn A mit B und B mit C verwandt ist, dann ist A mit C verwandt.
   * Durch ein neue oder bestehende Eigenschaft.

3) Transitivität (Transitivity):
   → Wenn A mit B verbunden ist, und B mit C verbunden ist, dann ist A mit C verbunden.
   * Mit der gleichen Eigenschaft.

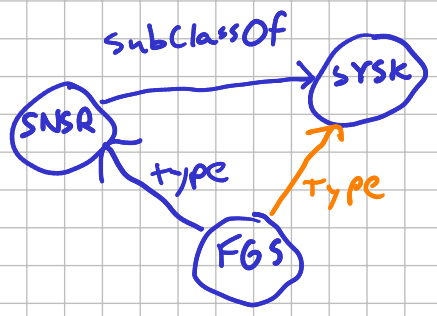
Bsp. → Klassen-Unterordnung

:Sensor rdfs:subClassOf :SystemKomponent

:FussgaengerSensor rdf:type :Sensor

⇒ Inferenz:

:FussgaengerSensor rdf:type :SystemK.



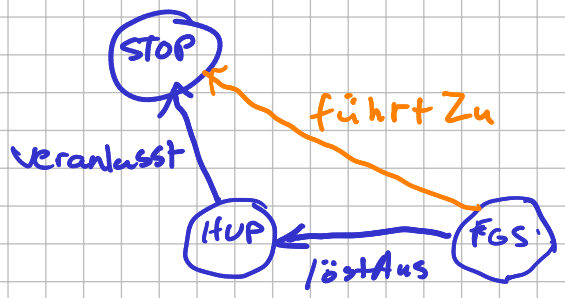
Bsp. Eigenschaftskette

Axiom

:führtZu owl:propertyChainAxiom (:löstAus :veranlasst)

:FussgaengerSensor :löstAus :Hupe

:Hupe :veranlasst :Stop



Inferenz

:FussgaengerSensor :führtZ :Stop

Bsp. Transitivität:

Axiom:

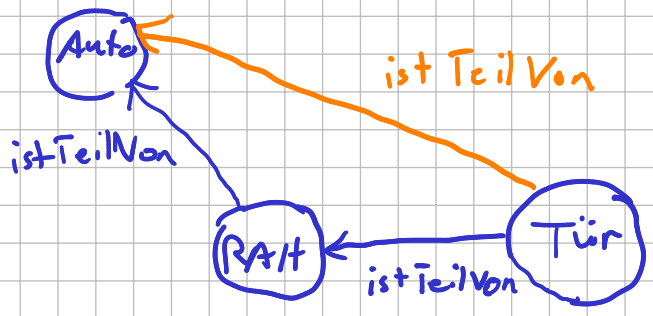
istTeilVon rdf:type owl:TransitiveProperty

:Tür :istTeilVon :Rahmen

:Rahmen :istTeilVon :AutonomesAuto

Inferenz

:Tür :istTeilVon :AutonomesAuto



KG Abfragen (Querying)

Während die Inferenz dazu dient, neues Wissen in eine KG zu scheffen, wird die <u>Abfrage</u> dazu verwendet, nutzlich Fakte aus der Graph zu extrahieren.

Abfragen beantworten Fragen wie:

- Wie viele der Komponenten des Autonomen Fahrzeug-steuerungssystems sind Sensoren?
- Führt der Fussgaengersensor in einem autonomen Auto dazu, dass das Auto anhält?

⇒ Obwohl es Methoden für die Abfrage in KGs gibt (z.B. SPARQL, Cypher, usw.), werden wir mit der eigentlichen Theorie der Suche in einem Graph befassen.

Abfrage als SuchProblem

Bei Graphen gibt es 2 wichtige Suchalgorithmen:

→ Breitensuche (breadth-First-search)

$\boxed{\text{BFS}}$ → FIFO Buffer

→ Tiefensuche (depth-First-search) $\boxed{\text{DFS}}$

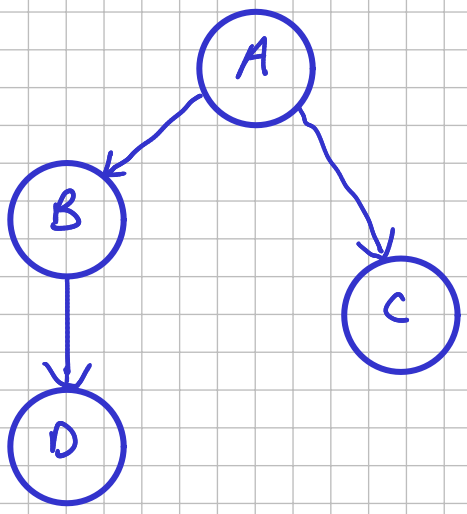
→ LIFO

BFS / DFS Query Ansatz

1. Bestimmen Sie einen Root Node
2. Definieren Sie Suchparameter
3. Definieren Sie Stopp-Bedingungen

Beispiel – BFS

1. Root Node A
2. Suchparameter: Anzahl der Nachbarn
3. Stop-Bedingung: kein weitere Nachbarn



Suchergebnisse:

- A hat 2 Nachbarn
- B hat 1 Nachbar
- C hat 0 Nachbarn

1) Der Root Node A wird in die FIFO Queue gestellt, während die Suchergebnisse ermittelt werden

2) A wird aus der FIFO entnommen, seine Suchergebnisse in der Liste

3) Die Nachbarn von Node A (B und C) werden in die FIFO aufgenommen

4) B wird aus der FIFO entnommen, seine Suchergebnisse in die Liste.

5) Die Nachbarn von B werden in die FIFO aufgenommen

6) C wird aus der FIFO entnommen, seine Suchergebnisse in die Liste

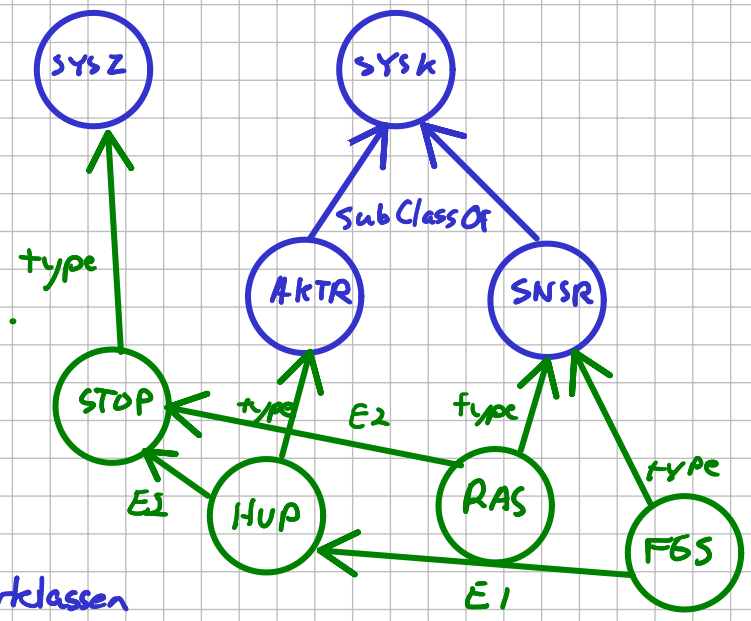
7) Stop-Bedingung erfüllt !

Beispiel - BFS Abfrage

Zur Beantwortung der Frage "Wie viele Komponenten des autonomen Fahrzeugsteuerungssystem sind Sensoren?",

⇒ Führen wir ein BFS mit Rückwärts-Traversal durch

1. RootNode: SYSK
2. SuchParameter:
   - Anzahl Nachbarn
   - Wenn Node kein Sensorklasse ist, übergehen
   - Wenn der Node ein Sensorklasse ist, sein Instanzen zählen
3. Stop-Bedingung
   - Alle Instanzen von Sensorklassen wurden gezählt.



Schritt 1

Der Root Node SYSK wird in die FIFO gestellt, während die Suchergebnisse ermittelt werden.

Suchergebnisse:

- SYSK hat 2 Nachbarn

Schritt 2

Die Nachbarn von SYSK werden in die FIFO aufgenommen.

Schritt 3

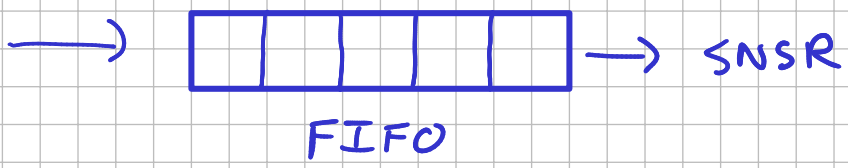
AKTR wird aus der FIFO entnommen und seine Suchergebnisse in die Liste

Suchergebnisse:

- AKTR ist kein Sensorklasse
  → übergehen

Schritt 4

SNSR wird aus der FIFO entnommen und seine Suchergebnisse in die Liste. Da unsere Suche die Anzahl der Sensoren (Instanzen der Klasse Sensor) erreicht haben, stoppen wir die Suche!



Suchergebnisse:

- SYSK hat 2 Nachbar
- AKTR ist keine Sensorklasse
- SNSR hat 2 Instanzen

Graphen - APL und Effizienz

Average Path Length (APL): Der Dürchschnitt der kürzesten Wege zwischen allen Node-Pairs i und j, für n nodes.

Für gerichtete Graphen ist dies definiert als:

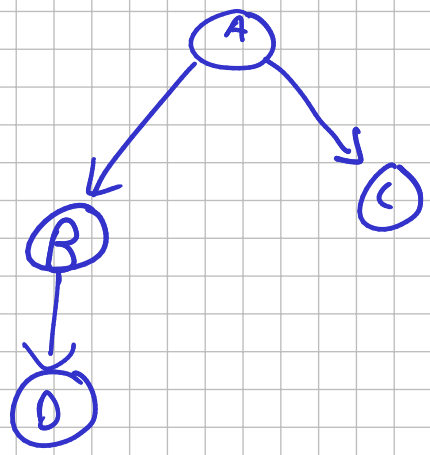
$$APL = \frac{1}{n(n-1)} \sum_{\substack{i,j \in V \\ i \neq j}} d(i,j).$$

* Nur die erreichbaren Node-Pairs zählen ⇓

$$\boxed{\frac{1}{R} \sum d(i,j)}$$

Bsp → $d(A,D)$ ist ein erreichbares Node-Pair

→ $d(D,A)$ ist kein erreichbares Node-Pair



$d(A,B) = 1$
$d(A,C) = 1$
$d(A,D) = 2$

$d(B,A) = x$
$d(B,C) = x$
$d(B,D) = 1$

$d(C,A) = x$
$d(C,\sim) = x$

$d(D,A) = x$
$d(D,\sim) = x$

$R = 4$

$$APL = \frac{1}{R}(1+1+2+1) = \frac{1}{4} \cdot 5$$

$$= \frac{5}{4} = 1.25$$

⇒ Die Global Effizien. (E_{Glob}): Durchschnitt der Umkehrung der kürzesten Wege zwischen alle Node-Pairs.

Für gerichtete Graphen ist es:

$$\frac{1}{n(n-1)} \sum_{\substack{i,j \in V \\ i \neq j}} \frac{1}{d(i,j)} \quad \boxed{\Rightarrow \frac{1}{R} \sum \frac{1}{d(i,j)}}$$

Da der Gesamtwirkungsgrad umgekehrt Proportional to APL ist, können wir die folgenden Nährung verwenden:

$$\boxed{E_{Glob} \approx \frac{1}{APL}}$$